

Multimodal RAG Pipeline

Design Conversation & Architecture Record

Project	E:\Rag_App\multimodal-rag
Scope	End-to-end RAG over complex PDFs returning text answers + exact images
Stack	PyMuPDF - Claude Opus 5 - Voyage AI - Qdrant - FastAPI - Streamlit
Status	Design approved, implementation pending
Document	Full conversation transcript with responses

Contents

- 1 Turn 1 - Project request and design overview
- 2 Design decisions (stack selection)
- 3 Confirmed stack and Phase 1 plan
- 4 Turn 2 - Step-by-step explanation of how it works
- 5 Turn 3 - Block diagrams with explanation
- 6 Turn 4 - Document generation request
- 7 Appendix - Quick reference

USER

Turn 1 - User

The original project request

E:\Rag_App\multimodal-rag Please access this folder. Before starting implementing the project, give me overview of designing and implement an end-to-end Multimodal RAG (Retrieval-Augmented Generation) pipeline capable of ingesting complex PDFs (containing text, embedded images, diagrams, and tables) and returning both synthesized text answers and the exact retrieved images to the user.

RESPONSE

Turn 1 - Response

Folder inspection and full design overview

Current state of the project folder

The folder is an empty scaffold: `.env` (empty), `.gitignore` (containing `.env`, `__pycache__`, `extracted_images`), an empty `requirements.txt`, and a git repository with zero commits. This is greenfield. Python is implied by the gitignore, so the assumption is Python plus Claude via the `anthropic` SDK.

1. The core problem

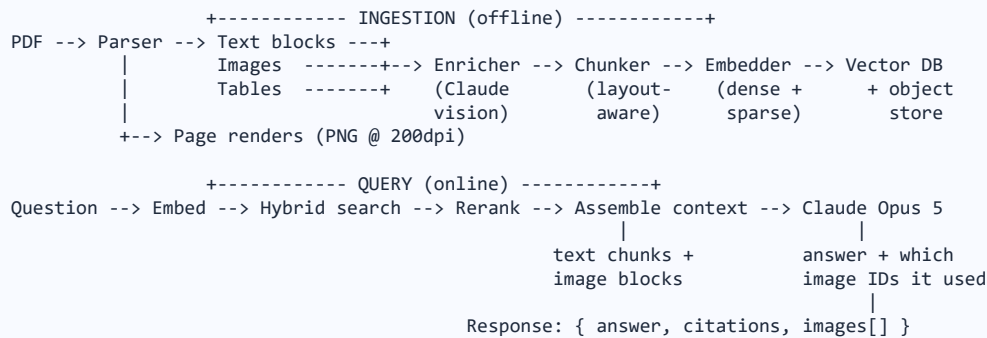
A normal RAG pipeline flattens a PDF into text and throws away everything that carries the actual information in technical documents: architecture diagrams, plots, screenshots and tables. This requirement adds a second twist - the answer must return **the exact retrieved image**, not a description of it. That means images are first-class retrievable objects with stable IDs, not decoration.

Three sub-problems fall out of this:

Sub-problem	Why it is hard
Extraction	PDFs have no semantic structure. Images are XObjects with no captions attached; tables are just lines and text runs.
Cross-modal retrieval	A text query ("show me the auth flow") must match a picture. Text and image embeddings must live in a comparable space - or you sidestep it entirely.
Grounded generation	Claude must answer from retrieved text and images, then cite which image it used so the exact file can be returned.

2. Architecture

High-level flow



3. Stage-by-stage design

3.1 Parsing - PyMuPDF as the spine

PyMuPDF (fitz) is the right primary tool - it gives text with bounding boxes, embedded image extraction, and page rasterisation from one library.

- **Text** - `page.get_text("dict")` returns blocks with bbox, font size and span info. Font size drives heading detection, which drives section-aware chunking.
- **Images** - `page.get_images()` extracts raw XObjects. Filter aggressively: drop anything under ~10k pixels, drop images with under 5% colour variance (logos, rules, background fills). Unfiltered extraction is the number one source of index pollution.
- **Tables** - `page.find_tables()` (PyMuPDF 1.23+) handles ruled tables well. For borderless tables it fails, and the fallback is to crop the region and let Claude vision transcribe it to Markdown.
- **Page renders** - rasterise every page at ~200 DPI. These are the safety net: if extraction misses a diagram, the page image still contains it, and they enable the ColPali-style fallback.

Optional upgrade: `unstructured` or a layout model (LayoutParser / DocLayNet) for cleaner region segmentation. Recommended to start without it - it is a heavy dependency and PyMuPDF covers most technical PDFs.

3.2 Enrichment - the key design decision

A raw image has no text to embed. You have to give it text. For each extracted image or table, run one Claude vision call that produces:

```
{
  "caption": "Sequence diagram of the OAuth2 authorization-code flow",
  "description": "Detailed prose: actors, arrows, labels, axis values...",
  "extracted_text": "all text visible inside the figure",
  "type": "diagram | chart | screenshot | table | photo",
  "keywords": ["oauth2", "pkce", "token exchange"]
}
```

Feed it the image plus surrounding page text (caption line, preceding paragraph) so it can resolve "Figure 4" references. Use structured outputs (`output_config.format`) so this is a validated object, not prose you have to regex. This is a batch job - run it through the Message Batches API at 50% cost, or Haiku 4.5 for simple figures with Opus 5 for dense diagrams.

The pivot of the design

The enriched text becomes the searchable surface; the image file itself is the payload you return.

3.3 Retrieval strategy - three options

	A. Summary-indexed (chosen)	B. Unified multimodal	C. Page-level visual
How	Embed the Claude-generated description; store image path as payload	Embed image pixels and text into one shared space (voyage-multimodal-3)	Embed whole page images as multi-vector patches; retrieve pages, not chunks
Pros	Cheap, debuggable, works with any text embedder, great recall on semantic queries	True cross-modal, no LLM enrichment cost, catches visual similarity	Zero parsing, no extraction bugs, state of the art on visually dense documents
Cons	Quality capped by caption quality; enrichment cost at ingest	Weaker on abstract queries; less mature tooling	Heavy index, needs GPU, cannot return sub-page crops
Verdict	Start here. Ship it, measure it.	Add later as a second retriever and fuse - genuinely complements A	Only for slide decks or scanned reports

Anthropic has no embeddings endpoint

Claude handles enrichment and generation; embeddings come from Voyage AI (voyage-3-large for text, voyage-multimodal-3 for option B) or a local bge/e5 model if you want zero external calls.

On top of whichever option is picked:

- **Hybrid search** - dense plus BM25/SPLADE, fused with RRF. Technical documents are full of exact identifiers (`ERR_TOKEN_EXPIRED`, Fig. 4.2) that dense retrieval alone misses.
- **Reranking** - voyage-rerank-2 or a cross-encoder over the top 50. The single highest-ROI addition to a RAG pipeline; expect a large precision jump.
- **Modality quotas** - do not let 20 text chunks crowd out every image. Reserve slots (for example top-8 text plus top-3 images) rather than taking a flat top-k.
- **Parent-document expansion** - retrieve on small chunks, send the enclosing section to the model.

3.4 Generation

Build the Claude message as interleaved content blocks:

```
[text: "Context from document X, page 12:"]
[text: <chunk>]
[text: "Figure F-0042 (page 12):"]
[image: <base64 or file_id of the actual image>]  <-- Claude SEES it
[text: <question>]
```

Claude Opus 5 with adaptive thinking. Key mechanics:

- **Citations** (`citations: {enabled: true}` on document blocks) return `page_location` automatically - grounded answers without prompt-engineering a citation format.
- **Prompt caching** on the system prompt and fixed instructions. Cache retrieved context too if the same document is queried repeatedly in a session.
- **Structured output** for the final envelope so `used_image_ids` is machine-readable: that array drives which images the API returns.
- **Streaming** - retrieval plus vision plus generation is a multi-second operation; stream so the UI is not dead.

3.5 Response contract

```
{
  "answer": "The auth flow uses PKCE. As shown in Figure 4...",
  "citations": [{"doc": "spec.pdf", "page": 12, "text": "..."}],
  "images": [
    {"id": "F-0042", "url": "/artifacts/spec/p12_img01.png",
     "page": 12, "bbox": [x0,y0,x1,y1], "caption": "OAuth2 sequence diagram",
     "score": 0.83, "cited_by_model": true}
  ]
}
```

`cited_by_model` matters: return everything retrieved (so the user can see near-misses) but flag what the answer actually leaned on.

4. Proposed repository structure

Repository layout

```
multimodal-rag/
+-- app/
|   +-- ingest/      parser.py extractor.py enricher.py chunker.py
|   +-- index/       embedder.py store.py bm25.py
|   +-- retrieve/     hybrid.py rerank.py fusion.py
|   +-- generate/     context.py claude.py prompts.py
|   +-- api/          main.py routes.py schemas.py
|   +-- core/         config.py models.py logging.py
+-- artifacts/       {doc_id}/page_{n}_img_{k}.png (served statically)
+-- data/            raw PDFs
+-- eval/            golden_qa.jsonl run_eval.py
+-- scripts/         ingest_pdf.py
+-- tests/
```

Endpoints: `POST /ingest` (async, returns job id), `GET /ingest/{job}`, `POST /query` (SSE stream), `GET /artifacts/{doc}/{file}`, `GET /health`.

Dependencies: anthropic, voyageai, pymupdf, qdrant-client, rank-bm25, fastapi, uvicorn, pydantic-settings, pillow, tenacity.

5. Build phases

- 1 **Extraction spike** - one PDF in, text plus filtered images plus tables on disk with metadata JSON. Eyeball the output; extraction quality determines everything downstream.
- 2 **Enrichment and indexing** - Claude vision descriptions, embed, load into the store.
- 3 **Retrieval** - hybrid plus rerank plus modality quotas. Test with hand-written queries before wiring the LLM.
- 4 **Generation** - context assembly, Claude call, structured response.
- 5 **API and streaming.**
- 6 **Eval harness** - 30 to 50 hand-labelled Q/A pairs with expected image IDs. Track recall@k, image-hit rate and answer faithfulness. Without this you are tuning blind.
- 7 **Hardening** - caching, batch ingest, dedup, cost telemetry.

6. Risks worth naming now

- **Extraction is where projects die.** Scanned PDFs need OCR (Tesseract or Claude vision on page renders). Vector-graphic diagrams are not XObjects at all - they are draw commands, and only page rasterisation catches them.

- **Caption quality is the retrieval ceiling** in approach A. Budget prompt-engineering time on the enricher, and evaluate captions directly.
- **Image token cost.** Full-resolution images on Opus 5 run up to ~4,800 tokens each. Cap images per answer (3 to 5) and downsample where fidelity is not needed.
- **Table fidelity.** Merged cells and multi-page tables break naive extraction. Markdown/HTML transcription via vision is more robust than coordinate-based reconstruction.
- **`.env` is empty** - `ANTHROPIC_API_KEY` plus an embedding provider key are needed before anything runs.

DECISION

Design Decisions

Three questions asked, three answers given

Question	Options offered	Decision
Which retrieval strategy first?	Summary-indexed / Unified multimodal / Both fused / Page-level visual	Summary-indexed
Where should vectors and artifacts live?	Qdrant local / Supabase pgvector / ChromaDB	Qdrant local
What should the delivery surface be?	FastAPI only / FastAPI + Streamlit / CLI only	FastAPI + Streamlit UI

RESPONSE

Response

Stack locked and Phase 1 scope

Stack locked

Layer	Choice
Parsing	PyMuPDF (text + bbox, image XObjects, find_tables, 200dpi page renders)
Enrichment	Claude vision - structured caption / description / extracted-text per image and table
Embeddings	Voyage voyage-3-large (text) - Anthropic has no embeddings endpoint
Vector store	Qdrant via Docker, named vectors + payload filtering, native sparse for hybrid
Rerank	voyage-rerank-2 over top-50
Generation	claude-opus-5, adaptive thinking, streaming, prompt caching, structured output for used_image_ids
Artifacts	Local disk under artifacts/{doc_id}/, served statically by FastAPI
Surface	FastAPI (REST + SSE) + Streamlit chat UI rendering retrieved images inline

`requirements.txt` will be: anthropic, voyageai, pymupdf, qdrant-client, fastapi, uvicorn[standard], streamlit, pydantic-settings, pillow, rank-bm25, tenacity, python-multipart.

Two blockers before anything can run

- 1 `.env` is empty - needs `ANTHROPIC_API_KEY` and `VOYAGE_API_KEY`. The `.env.example` and config loader will be written; the keys must be supplied.
- 2 **No PDFs anywhere in the project.** Phase 1 is an extraction spike, and extraction quality against real documents is what determines whether the summary-indexed approach holds up. A representative PDF (ideally one with diagrams and a borderless table) should go in `data/`.

Phase 1 scope - scaffold plus extraction spike, no LLM calls

- Package structure, `config.py` (pydantic-settings), `models.py` (the Document / TextBlock / ImageAsset / TableAsset schemas everything else depends on), `requirements.txt`, `.env.example`

- `ingest/parser.py` - page iteration, text blocks with bbox plus font-size heading detection
- `ingest/extractor.py` - image XObject extraction with the noise filters (size plus colour variance), table detection, page rasterisation
- `scripts/ingest_pdf.py` - CLI that dumps `artifacts/{doc_id}/` plus a `manifest.json` for inspection

That gives something inspectable before a single token is spent on enrichment.

USER

Turn 2 - User

Request for a plain-language walkthrough

explain me each steps and it's functioning properly in simple way, then I start to implement

RESPONSE

Turn 2 - Response

How the whole thing works, step by step

Walking one PDF through the system, then one question. Running example: `payments-api-spec.pdf`, 40 pages, with an OAuth sequence diagram on page 12 and a rate-limit table on page 18.

The one-sentence version

Ingestion turns a PDF into a searchable library where every paragraph, picture and table is a separate labelled item; querying finds the handful of items relevant to a question and hands them to Claude to answer from.

Part A - Ingestion (once per PDF, offline)

Step 1 - Parse the pages

In	The PDF file.
What happens	PyMuPDF opens it and walks page by page. For each page it pulls out text with coordinates - not one blob of text, but a list of blocks, each with its position on the page and its font size.
Why it matters	Font size tells you "3.2 Token Refresh" is a heading and the paragraph under it is body text. Position tells you which caption sits under which picture. Plain text extraction throws both away, and then you cannot tell a heading from a footnote.
Out	A list like {text: "3.2 Token Refresh", page: 12, bbox: [72, 90, 300, 108], font_size: 14}.
Healthy when	Headings are detected as headings, and reading order is sane on two-column pages.

Step 2 - Extract the pictures

PDFs store images as embedded objects. PyMuPDF pulls each one out as a real PNG file and records which page it came from and where on that page it sat.

Then the filtering, which matters more than the extraction: a 40-page PDF might yield 200 "images", of which 180 are the company logo repeated in every header, horizontal rule lines, and background shading. Two filters cut them:

- **Size** - anything under ~10,000 pixels is decoration.
- **Colour variance** - an image that is 98% one colour is a rule, a fill, or a bullet, not a diagram.

Separately, every page is also photographed as a whole-page PNG at 200 DPI. That is a safety net: some diagrams are not stored as images at all, they are drawing instructions (lines and shapes), so extraction finds nothing - but the page photo still contains the diagram.

Out

artifacts/payments-api-spec/page12_img01.png - the actual OAuth diagram, on disk, with a stable ID.

Healthy when

You open the artifacts folder and see ~15 real figures, not 200 logos.

Step 3 - Find the tables

PyMuPDF detects tables by finding the ruling lines and the grid of text inside them, then gives you rows and columns. This works well when the table has visible borders. Borderless tables (just whitespace alignment) defeat it - for those, the page region is cropped as an image and Claude reads it in the next step.

Healthy when

The rate-limit table on page 18 comes back with its columns intact, not scrambled into one line.

Step 4 - Describe the pictures (the clever bit)

The core problem

You cannot search a picture. Search works by comparing meaning between pieces of text. A PNG has no text in it.

So we give each picture text. Every extracted image and table is sent to Claude with vision, along with the text that surrounds it on the page (the caption line, the paragraph above it), and Claude writes a description:

```
caption: "OAuth2 authorization-code flow with PKCE"
description: "Sequence diagram with four actors: Client App, User Agent,
             Auth Server, Resource Server. Shows the initial authorization
             request, the code challenge, token exchange, and the retry
             path when the access token has expired..."
extracted_text: "code_challenge, code_verifier, 401 TOKEN_EXPIRED, /oauth/refresh"
type: "diagram"
```

Two things to notice:

- The surrounding page text is passed so Claude can resolve references. Without it, a diagram labelled only "Figure 4" is meaningless. With it, Claude knows Figure 4 is the auth flow.
- `extracted_text` captures the labels inside the picture. That is how a search for `TOKEN_EXPIRED` can find a diagram - the string only exists as pixels in the image, but now it exists as text in the index.

From here on

The picture is represented by its description. The description gets searched; the PNG gets returned.

Healthy when

You read 10 descriptions cold, without seeing the images, and you can tell what each picture shows.

Step 5 - Chunk the text

You cannot index a whole 40-page document as one item - a search would either match everything or nothing. So it is cut into pieces of roughly 500 to 800 words.

But the cuts are on **section boundaries**, not every N characters. Chopping mid-sentence at character 500 splits an explanation across two chunks, and then neither chunk makes sense on its own. Using the headings from Step 1, each chunk is a coherent unit, and it carries its heading trail with it: "3. Authentication > 3.2 Token Refresh". That context helps both retrieval and the final answer.

Out

~120 text chunks + ~15 image descriptions + ~6 table descriptions. About 140 searchable items, each with a page number and, for images, a path to the PNG.

Step 6 - Turn everything into numbers (embeddings)

Each piece of text goes to Voyage's embedding model, which returns a list of ~1,000 numbers - a "vector". The useful property: **texts with similar meaning produce similar vectors**. "how do I refresh an expired token" and "token renewal procedure" land close together numerically, even though they share almost no words.

That is what makes semantic search work. Keyword search would miss that pair entirely. Note that image descriptions get embedded exactly like text chunks - they are just text at this point. That is the whole trick of summary-indexing.

Step 7 - Store it

Everything goes into Qdrant. Each stored record holds the vector (for meaning-based search), the original text (to show the model later), and metadata: `doc_id`, `page`, `type` (text/image/table), and for images the path to the PNG. The PNG files themselves stay on disk; Qdrant just stores the path.

Two indexes get built, not one:

- **Dense** (the vectors) - finds things by meaning.
- **BM25 / keyword** - finds exact strings. Technical documents are full of identifiers like `ERR_TOKEN_EXPIRED` or `Figure 4.2`, and vector search is genuinely bad at exact-token matching. You need both.

Ingestion is done

The PDF is now a searchable library.

Part B - Answering a question (per query, live)

Question: "How does the auth flow handle expired tokens?"

Step 8 - Embed the question

Same model as Step 6 - the question becomes a vector. It has to be the same model, or the numbers are not comparable.

Step 9 - Search, two ways at once

- **Dense search**: find the ~50 stored vectors closest to the question's vector. This catches the token-refresh section even if it never says "expired".
- **Keyword search**: find items containing `auth`, `expired`, `token`. This catches the exact error code.

The two result lists are merged with RRF - items ranked highly by both retrievers float to the top.

The modality quota

Reserve slots by type, for example top 8 text plus top 3 images. Without this, text chunks win every time on raw score and images never surface - which would defeat the whole point of the project.

Step 10 - Rerank

The first search was fast and approximate - it compared pre-computed vectors. Reranking is slow and precise: a model reads the question and each candidate together and scores actual relevance.

This is the single highest-impact step for answer quality. Fast search gets you in the neighbourhood; reranking picks the right house. A chunk that merely mentions "token" drops; the one that explains the refresh path rises. Output: top ~10, correctly ordered.

Step 11 - Build the message for Claude

```
"Context from payments-api-spec.pdf, page 11:"  
[text of the token-refresh section]  
  
"Figure F-0042 from page 12:"  
[the actual PNG of the OAuth diagram]      <-- Claude sees the real image  
  
"Question: How does the auth flow handle expired tokens?"
```

Important

Claude receives the real image, not the description. The description was only ever a search key to find the image. Once found, the model looks at the diagram itself and can read details the caption never mentioned.

Step 12 - Generate

Claude Opus 5 reads the text, looks at the images, and answers. Two settings do real work:

- **Citations** - Claude marks which source each claim came from, with page numbers, automatically.
- **Structured output** - the response comes back as a fixed shape including `used_image_ids: ["F-0042"]`, so the code knows which images the answer actually relied on, rather than guessing.

Step 13 - Return

```
{  
  "answer": "When the access token expires the resource server returns  
            401 TOKEN_EXPIRED. The client then calls /oauth/refresh with  
            the stored refresh token... (see Figure 4)",  
  "citations": [{"page": 11}, {"page": 12}],  
  "images": [  
    {"id": "F-0042", "url": "/artifacts/payments-api-spec/page12_img01.png",  
      "page": 12, "caption": "OAuth2 flow with PKCE", "cited_by_model": true}  
  ]  
}
```

The Streamlit UI renders the answer text and displays that PNG inline underneath. **That image is the exact file extracted from page 12 in Step 2** - same bytes, not a regeneration, not a description.

How you will know each stage is healthy

Stage	The check	Bad sign
Parse	Headings detected; two-column pages read in order	Text scrambled across columns
Extract images	~15 real figures in artifacts/	200 files, mostly logos
Tables	Columns preserved	Everything on one row
Describe	Read descriptions blind - can you tell what each shows?	Vague: "a diagram with boxes"

Stage	The check	Bad sign
Chunk	Each chunk readable standalone	Sentences cut in half
Embed	"token refresh" and "renew credentials" score as similar	Random similarity scores
Search	Query for a known figure - does it come back top-5?	Right answer at rank 30
Rerank	Top-3 are all genuinely relevant	Order unchanged from search
Generate	Answer cites the right pages	Confident answer, wrong pages

The eval harness (Phase 6) automates the last three: ~40 hand-written questions, each labelled with the pages and image IDs that should come back. Then any change to chunk size, filters or prompts is measurable instead of vibes.

Where the real difficulty is

Not in the retrieval mathematics - that is largely solved libraries. It is in Steps 2 and 4:

- If extraction pulls out garbage, you index garbage. Filters need tuning against your PDFs.
- If the descriptions are vague, retrieval is capped no matter how good the embedding model is. In the summary-indexed design, **caption quality is the ceiling on image recall**. That is the one prompt worth iterating on properly.

Everything else is plumbing.

USER

Turn 3 - User

Request for block diagrams

give me Explanation with block diagram. Give me block diagram with explanation

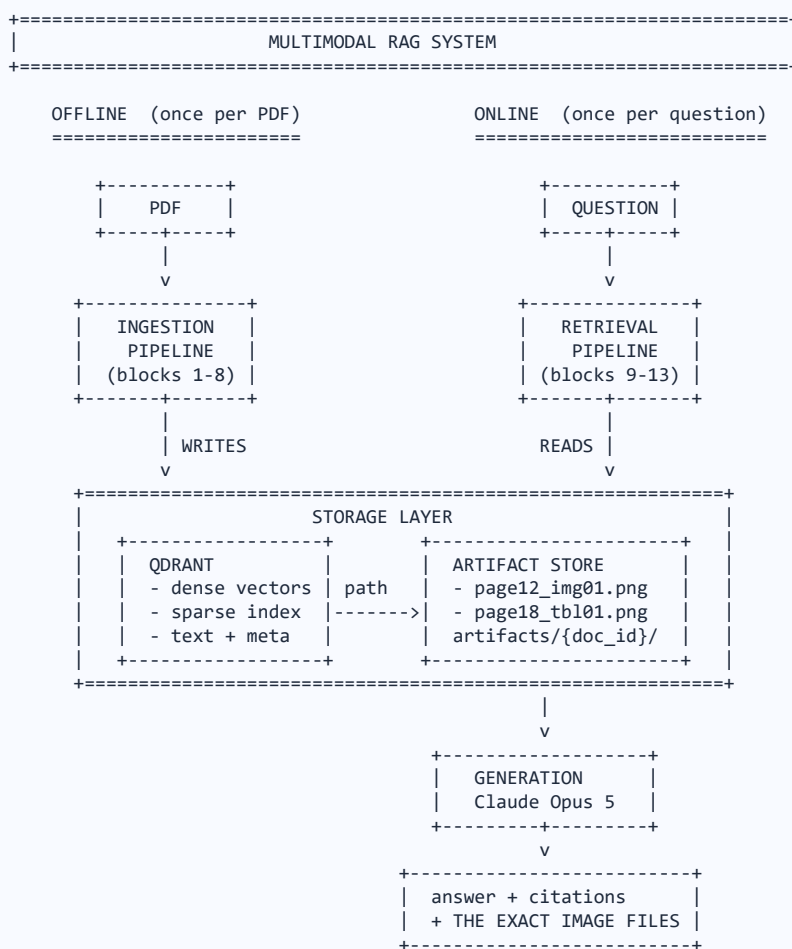
RESPONSE

Turn 3 - Response

Five block diagrams with per-block explanation

Diagram 1 - System overview

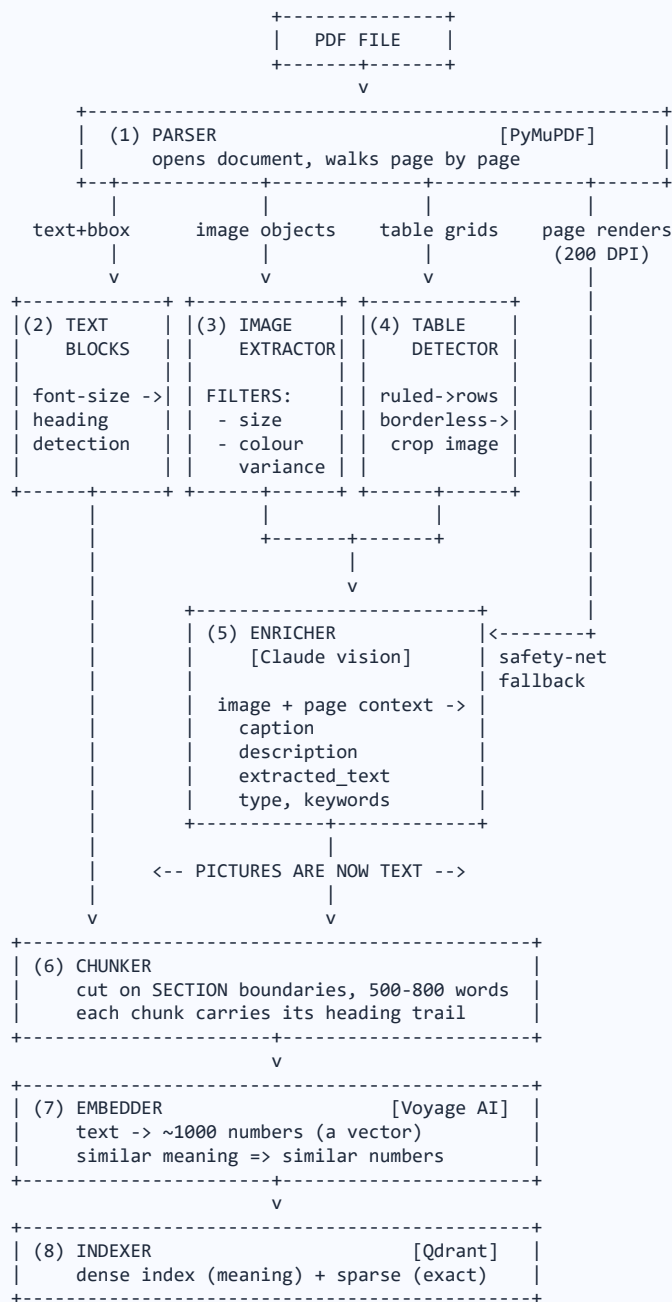
Two pipelines joined only by storage



Reading it: two pipelines that never talk to each other directly. They communicate only through the storage layer. Ingestion is slow, expensive, and runs once. Retrieval is fast and runs on every question. Qdrant holds searchable numbers; the artifact store holds the actual PNG bytes you eventually return. Qdrant only stores the file path - it never stores an image.

Diagram 2 - Ingestion pipeline (blocks 1 to 8)

Three lanes converging at the enricher



What each ingestion block does

#	Block	In	Out	Job
1	Parser	PDF	pages	Opens the file, exposes text/images/tables/renderers per page
2	Text blocks	raw text runs	blocks + heading level	Keeps position and font size so structure survives
3	Image extractor	embedded objects	PNG files	Pulls real images out; filters kill logos and rule lines
4	Table detector	ruling lines + text grid	rows, or a crop	Structured tables when ruled, image fallback when not

#	Block	In	Out	Job
5	Enricher	image + surrounding text	a text description	Makes pictures searchable
6	Chunker	text + descriptions	~140 chunks	Right-sized, self-contained units
7	Embedder	chunk text	vector	Converts meaning into comparable numbers
8	Indexer	vectors + metadata	populated DB	Two indexes: semantic and keyword

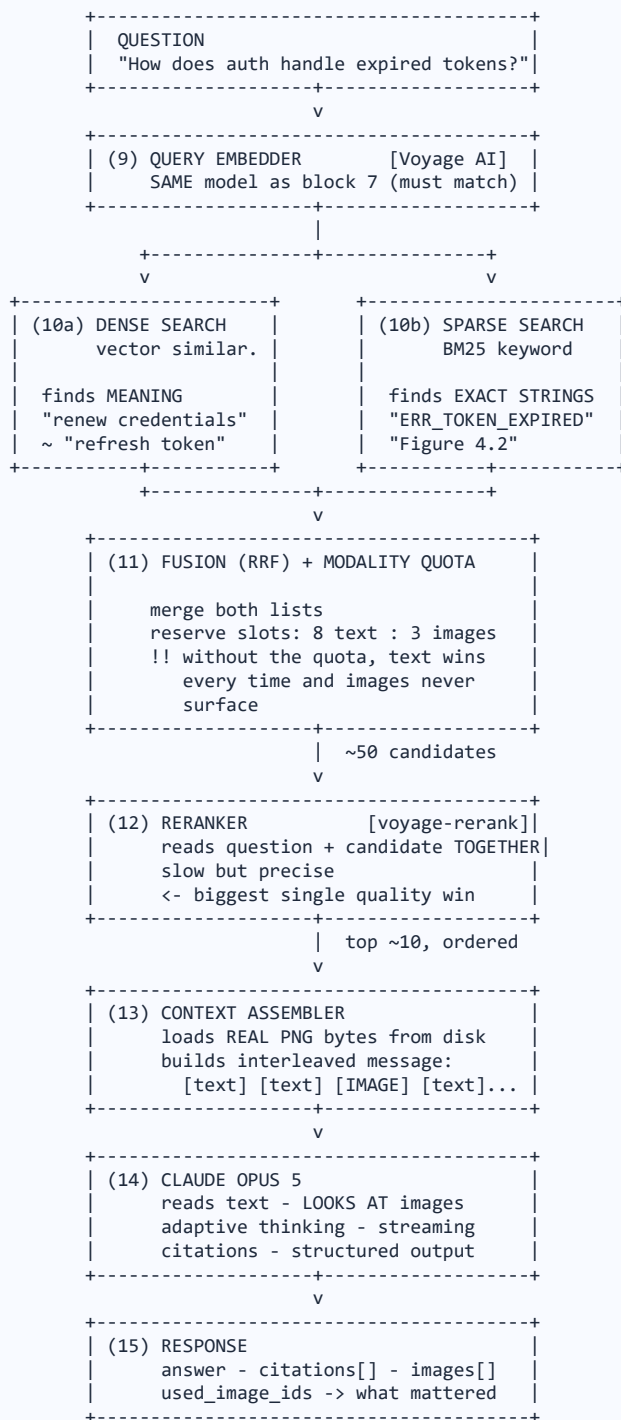
The critical junction is block 5

Notice where the three lanes converge: text goes straight down, but images and tables have to pass through the enricher first. That is the pivot of the whole design - a PNG cannot be searched, so Claude writes text about it, and from block 6 onward an image is treated exactly like any other paragraph. The PNG itself sits untouched on disk, waiting to be handed back at the very end.

Also note the path on the right: page renders bypass blocks 1 to 4 entirely and feed block 5 directly. That is the safety net for diagrams drawn as vector shapes, which produce zero extractable images.

Diagram 3 - Query pipeline (blocks 9 to 15)

Dual search, fusion, rerank, generate



What each query block does

#	Block	In	Out	Job
9	Query embedder	question text	vector	Puts the question in the same number-space as the index
10	Dual search	query vector + terms	~50 candidates	Meaning search and exact-string search, in parallel

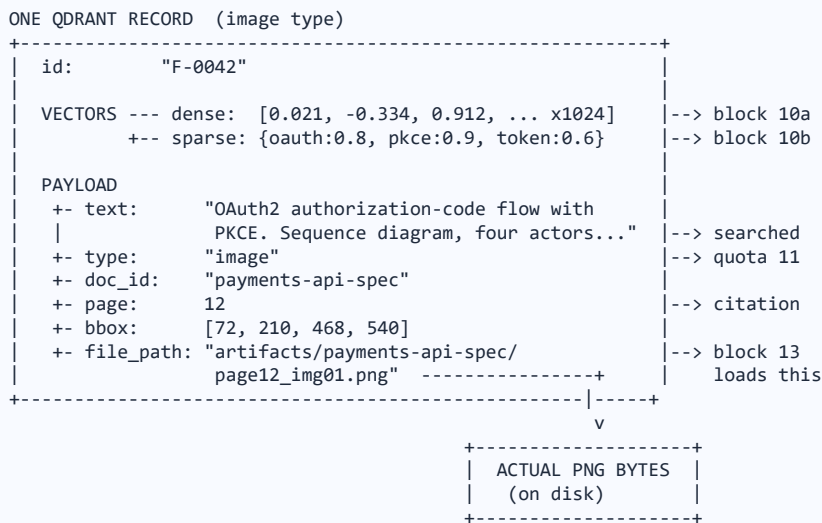
#	Block	In	Out	Job
11	Fusion + quota	two ranked lists	one ranked list	Merges them; guarantees images get seats at the table
12	Reranker	50 candidates	top 10	Re-scores properly; fixes approximate search's mistakes
13	Assembler	top 10 + PNG paths	Claude message	Swaps descriptions back out for real image bytes
14	Claude	text + images + question	answer	Reasons over both modalities, cites sources
15	Response	model output	JSON	Answer, page citations, and the image files

The critical junction is block 13

This is where the trick from block 5 reverses. During search, an image was represented by its description. At block 13 that description is discarded and the actual PNG is loaded from disk and put into the message. So Claude never sees the caption - it sees the diagram, and can read details the caption never mentioned. Then the same file is returned to the user.

Diagram 4 - What a stored record looks like

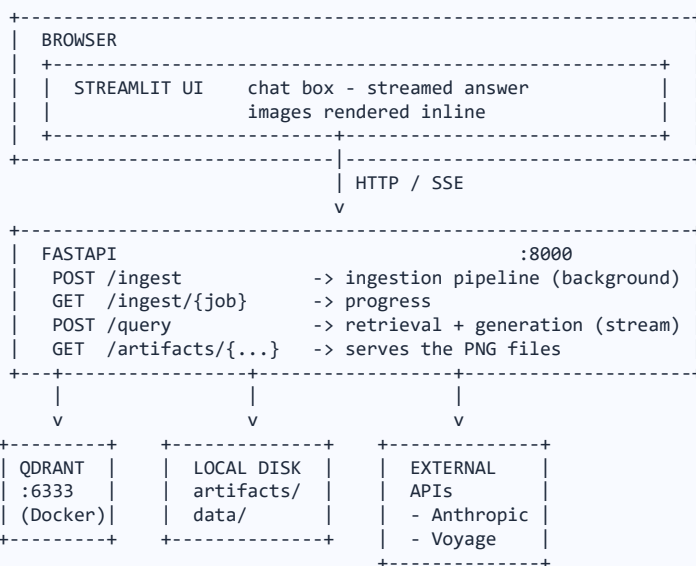
One Qdrant record of type image



The record is a **pointer with a searchable label**. The label (**text**) is what search matches on. The pointer (**file_path**) is what gets dereferenced at block 13 and returned at block 15. Text chunks use the identical shape - **type: "text"**, no **file_path**.

Diagram 5 - Runtime components

What runs where



Everything except the two API providers runs on your machine. Qdrant is one **docker run**; artifacts are plain files FastAPI serves statically; Streamlit talks to FastAPI over HTTP so the UI can be swapped later without touching the pipeline.

The two ideas the diagrams are really showing

- 1 **Images become text at block 5, and become images again at block 13.** Search operates on descriptions; answering operates on pixels; the user receives the original file. Every design choice downstream follows from that round trip.
- 2 **The pipelines are decoupled by storage.** Ingestion never calls retrieval. That means you can re-ingest a document, swap the embedding model, or rebuild the index without touching the query path - and you can tune retrieval without re-parsing 40 pages every time.



USER

Turn 4 - User

Document export request

Add all conversation with response in the PDF document, and word document with colourful design and give

RESPONSE

Turn 4 - Response

This document

The full conversation was transcribed into a structured content module and rendered to both PDF and DOCX with a shared colour design system. Environment work required: Python 3.12 was located via the `py` launcher (the `python` command on PATH is the Microsoft Store stub and does not work), and `python-docx`, `reportlab` and `fonttools` were installed.

Font coverage was verified before rendering, because the block diagrams use box-drawing characters that many PDF base fonts lack. Consolas covers the full box-drawing and circled-digit range; Calibri covers the prose set. The diagrams in this document use an ASCII-safe transliteration so they render identically in both formats and in any viewer.

Appendix - Quick reference

The 15 blocks at a glance

Phase	#	Block	One-line function
Ingest	1	Parser	Walk pages, expose text/images/tables/renders
Ingest	2	Text blocks	Heading detection from font size
Ingest	3	Image extractor	Extract PNGs, filter out logos and rules
Ingest	4	Table detector	Ruled tables to rows, borderless to crops
Ingest	5	Enricher	Claude vision writes searchable text for pictures
Ingest	6	Chunker	Section-aware 500-800 word units
Ingest	7	Embedder	Text to vectors (Voyage)
Ingest	8	Indexer	Dense + sparse indexes in Qdrant
Query	9	Query embedder	Question to vector, same model
Query	10	Dual search	Dense (meaning) + BM25 (exact)
Query	11	Fusion + quota	RRF merge, reserve image slots
Query	12	Reranker	Cross-encoder precision pass
Query	13	Assembler	Load real PNGs, interleave with text
Query	14	Claude Opus 5	Answer from text + images
Query	15	Response	answer + citations + image files

Decisions locked

Retrieval strategy	Summary-indexed (Claude vision captions, embedded as text)
Vector store	Qdrant local via Docker, artifacts on disk
Delivery surface	FastAPI (REST + SSE) plus Streamlit UI
Generation model	claude-opus-5 with adaptive thinking and streaming
Embedding model	voyage-3-large; reranking with voyage-rerank-2

Immediate next actions

- 1 Add `ANTHROPIC_API_KEY` and `VOYAGE_API_KEY` to `.env`.
- 2 Place a representative PDF (with diagrams and at least one borderless table) in `data/`.
- 3 Run Phase 1 - scaffold plus parser and extractor, no LLM calls - and inspect `artifacts/{doc_id}/` plus `manifest.json` to judge extraction quality.
- 4 Only after extraction looks clean, proceed to Phase 2 (enrichment and indexing).

The rule to remember

Extraction quality caps everything downstream. Spend the time in blocks 3 and 5 before touching retrieval tuning - no amount of reranking rescues a badly extracted, badly captioned index.